

1. RL MDP



Instructors: Joun Won

Joun Won

2026 Spring

Contents

1	RL MDP	3
1.1	Why RL	3
1.2	What is RL	3
1.3	Difference between RL, SL, and USL	4
1.4	Components of RL	5
1.5	Markov Decision Process (MDP)	6
1.5.1	Motivation	6
1.5.2	The Markov Property	6
1.5.3	Formal Definition of an MDP	7
1.5.4	The Discount Factor and Convergence of Return	8
1.5.5	Trajectories and Episodes	8
1.5.6	Value Functions Under an MDP	9
1.5.7	Example: A Simple MDP	9
1.5.8	Summary	10

1 RL MDP

1.1 Why RL

Why do we use Reinforcement Learning?

- When an accurate system model is unknown or hard to build.
- Nonlinearity + high-dimensional systems.
- Robustness to model mismatch.
- Optimizing performance metrics that are hard to define explicitly.
- Similarity to online adaptation. (adaptive control)

1.2 What is RL

Reinforcement Learning: a type of machine learning where an AI program learns to make decisions by trying different actions and seeing what happens.

In reinforcement learning, autonomous agents learn to perform a task by trial and error in the absence of any guidance from a human user.

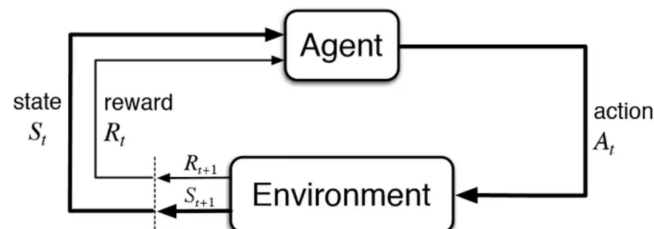


Figure 1.1: Agent-environment interaction loop in reinforcement learning. At each timestep t , the agent observes the state S_t and reward R_t , takes an action A_t , and the environment transitions to a new state S_{t+1} with reward R_{t+1} .

1.3 Difference between RL, SL, and USL

Machine learning paradigms are commonly divided into three broad categories, distinguished primarily by *what kind of feedback* the learning algorithm receives from the data or environment.

Supervised Learning. In supervised learning, the algorithm is given a dataset of labeled examples $\{(x_i, y_i)\}_{i=1}^N$, where x_i is an input and y_i is the corresponding “correct answer” (label). The goal is to learn a mapping $f : X \rightarrow Y$ that generalizes well to unseen inputs. Feedback is *instructive*: for every input, the model is told exactly what the correct output should have been, regardless of what it actually predicted.

- **Feedback type:** Correct label provided for every example.
- **Goal:** Minimize prediction error (e.g., classification error, regression loss).
- **Examples:** Image classification, spam detection, house price prediction.

Unsupervised Learning. In unsupervised learning, the algorithm is given only inputs $\{x_i\}_{i=1}^N$, with no labels at all. The goal is to discover hidden structure, patterns, or groupings within the data itself, rather than to predict a specific output.

- **Feedback type:** None — no labels or rewards are provided.
- **Goal:** Discover structure (e.g., clusters, latent factors, density estimates).
- **Examples:** Clustering (k-means), dimensionality reduction (PCA), anomaly detection.

Reinforcement Learning. In reinforcement learning (RL), an agent interacts with an environment over a sequence of timesteps. At each timestep t , the agent observes a state S_t , takes an action A_t , and receives a scalar reward R_{t+1} along with the next state S_{t+1} (see Figure 1.1). The agent is never told the “correct” action to take; it must discover good actions through trial and error, using the reward signal as *evaluative* rather than instructive feedback.

- **Feedback type:** Scalar reward signal, often delayed, indicating how good an action was — not what the correct action would have been.
- **Goal:** Learn a policy $\pi(a | s)$ that maximizes cumulative (expected discounted) reward over time.
- **Examples:** Game playing (AlphaGo, Atari), robotic control, resource allocation.

Key distinction: evaluative vs. instructive feedback. The core difference between RL and supervised learning is the *nature* of the feedback:

- Supervised learning: *instructive* feedback — the label directly tells the model what it should have output.

- Reinforcement learning: *evaluative* feedback — the reward tells the agent how good its action was, but not what the best action would have been. The agent must actively explore to find better actions.

Additionally, RL problems are inherently *sequential*: an action taken now can affect future states and future rewards (delayed reward / credit assignment problem), whereas supervised and unsupervised learning typically treat each data point as independent and identically distributed (i.i.d.).

	Supervised	Unsupervised	Reinforcement
Data	Labeled (x_i, y_i)	Unlabeled x_i	Sequential (S_t, A_t, R_t)
Feedback	Correct label (instructive)	None	Scalar reward (evaluative)
Feedback timing	Immediate, per example	—	Often delayed
Goal	Minimize prediction error	Discover structure	Maximize cumulative reward
Data assumption	i.i.d. samples	i.i.d. samples	Sequential, non-i.i.d. (depends on agent's own actions)
Exploration needed?	No	No	Yes

Table 1.1: Comparison of supervised, unsupervised, and reinforcement learning.

1.4 Components of RL

Definition 1. State (S_t): A representation of the environment at time t that the agent uses to decide its next action. It may be the full state of the environment (fully observable) or a partial observation (partially observable).

Definition 2. Action (A_t): A choice made by the agent at time t from a set of available actions $\mathcal{A}$ (or $\mathcal{A}(S_t)$, if the action set depends on the current state), which affects the environment and determines the transition to the next state.

Definition 3. Reward (R_t): A scalar feedback signal $R_t \in \mathbb{R}$ received from the environment after taking an action, indicating how good or bad that action was at that timestep. The agent's objective is to maximize the cumulative reward over time, not the immediate reward alone.

Definition 4. Policy (π): A mapping from states to actions that defines the agent's behavior. It can be deterministic, $\pi(s) = a$, or stochastic, $\pi(a | s) = P(A_t = a | S_t = s)$.

Definition 5. Model: The agent's (optional) representation of the environment's dynamics, consisting of a transition function $P(s' | s, a)$ predicting the next state, and a reward function $R(s, a)$ predicting the expected reward. Agents that use a model are called *model-based*; those that do not are *model-free*.

Definition 6. Environment: Everything external to the agent that it interacts with. The environment receives the agent's action A_t , transitions to a new state S_{t+1} , and emits a reward R_{t+1} (see Figure 1.1).

Definition 7. Agent: The learner and decision-maker that observes states, selects actions according to its policy, and updates its policy (and/or value function or model) based on the rewards it receives, with the goal of maximizing cumulative reward.

1.5 Markov Decision Process (MDP)

1.5.1 Motivation

So far, we have informally introduced the components of RL: state, action, reward, policy, return, and value functions. The *Markov Decision Process* (MDP) is the mathematical framework that formalizes all of these components into a single, well-defined structure. Almost all RL problems can be formalized as MDPs (or generalizations thereof), which is why MDPs are considered the foundational model of sequential decision-making under uncertainty.

1.5.2 The Markov Property

Definition 8. Markov Property: A state S_t is said to be Markov if

$$P(S_{t+1} | S_t) = P(S_{t+1} | S_1, S_2, \dots, S_t).$$

That is, the future is conditionally independent of the past given the present state.

Intuitively, the current state contains all the information from history that is relevant for predicting the future. This is often called the **memoryless property**.

Lemma 1.1: Trajectory Factorization under the Markov Property

If every state S_t in a sequence $S_0, S_1, \dots, S_T$ satisfies the Markov property, then the joint probability of the trajectory factorizes as

$$P(S_0, S_1, \dots, S_T) = P(S_0) \prod_{t=0}^{T-1} P(S_{t+1} | S_t).$$

Proof 1.1: Trajectory Factorization under the Markov Property

By the chain rule of probability, for any sequence of random variables,

$$P(S_0, S_1, \dots, S_T) = P(S_0) \prod_{t=0}^{T-1} P(S_{t+1} \mid S_0, S_1, \dots, S_t).$$

By the Markov property (Definition 8), each conditional term satisfies

$$P(S_{t+1} \mid S_0, \dots, S_t) = P(S_{t+1} \mid S_t).$$

Substituting this into the chain rule expression above yields

$$P(S_0, S_1, \dots, S_T) = P(S_0) \prod_{t=0}^{T-1} P(S_{t+1} \mid S_t),$$

as claimed. ■

□

Remark. This lemma is the reason the Markov property is so powerful computationally: instead of storing an exponentially growing joint distribution over the full history, we only ever need the one-step transition probabilities $P(S_{t+1} \mid S_t)$.

When extended to include actions and rewards, the Markov property becomes

$$P(S_{t+1}, R_{t+1} \mid S_t, A_t) = P(S_{t+1}, R_{t+1} \mid S_1, A_1, R_1, \dots, S_t, A_t),$$

which is the core assumption underlying the MDP framework.

1.5.3 Formal Definition of an MDP

Definition 9. Markov Decision Process: An MDP is a 5-tuple

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' \mid s, a)$ is the state transition probability function, $R(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor.

Interpreting each component.

- $\mathcal{S}, \mathcal{A}$: what is possible.
- P : how the world evolves (environment dynamics).
- R : what is desirable (the objective).
- γ : how much the agent values the future.

1.5.4 The Discount Factor and Convergence of Return

Definition 10. Return: $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$.

A natural question is whether G_t is even well-defined (finite) for continuing tasks with an infinite horizon. The following theorem answers this.

Theorem 1.2: Boundedness of the Discounted Return

Suppose rewards are bounded, i.e., $|R_t| \leq R_{\max}$ for all t , and $\gamma \in [0, 1)$. Then the discounted return G_t is finite and satisfies

$$|G_t| \leq \frac{R_{\max}}{1 - \gamma}.$$

Proof 1.2: Boundedness of the Discounted Return

By the triangle inequality and the bound on rewards,

$$|G_t| = \left| \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \right| \leq \sum_{k=0}^{\infty} \gamma^k |R_{t+k+1}| \leq \sum_{k=0}^{\infty} \gamma^k R_{\max} = R_{\max} \sum_{k=0}^{\infty} \gamma^k.$$

Since $0 \leq \gamma < 1$, the geometric series converges:

$$\sum_{k=0}^{\infty} \gamma^k = \frac{1}{1 - \gamma}.$$

Therefore,

$$|G_t| \leq \frac{R_{\max}}{1 - \gamma},$$

which is finite. ■

□

Remark. This is precisely why $\gamma < 1$ is required for continuing tasks: without discounting, an infinite sum of bounded (but nonzero) rewards would diverge. For episodic tasks, the sum is automatically finite (terminates at the terminal state), so $\gamma = 1$ is sometimes permissible.

1.5.5 Trajectories and Episodes

Definition 11. Episodic Task: A task that breaks into finite episodes, each ending in a terminal state.

Definition 12. Continuing Task: A task with no natural endpoint; interaction continues indefinitely.

1.5.6 Value Functions Under an MDP

Definition 13. State-Value Function: $V_\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s]$.

Definition 14. Action-Value Function: $Q_\pi(s, a) = \mathbb{E}_\pi [G_t \mid S_t = s, A_t = a]$.

The two value functions are not independent; the following theorem makes their relationship precise.

Theorem 1.3: Relation between V_π and Q_π

For any policy π and any state $s \in \mathcal{S}$,

$$V_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) Q_\pi(s, a).$$

Proof 1.3: Relation between V_π and Q_π

By Definition 13 and the law of total expectation, conditioning on the action taken at time t ,

$$V_\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s] = \sum_{a \in \mathcal{A}} P(A_t = a \mid S_t = s) \mathbb{E}_\pi [G_t \mid S_t = s, A_t = a].$$

By definition of the policy, $P(A_t = a \mid S_t = s) = \pi(a \mid s)$, and by Definition 14, $\mathbb{E}_\pi [G_t \mid S_t = s, A_t = a] = Q_\pi(s, a)$. Substituting gives

$$V_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) Q_\pi(s, a),$$

as claimed. ■

□

The goal of RL, restated. The agent seeks an optimal policy

$$\pi^* = \arg \max_{\pi} V_\pi(s), \quad \forall s \in \mathcal{S}.$$

1.5.7 Example: A Simple MDP

Consider three states $\mathcal{S} = \{\text{Cool}, \text{Warm}, \text{Overheated}\}$ and two actions $\mathcal{A} = \{\text{Slow}, \text{Fast}\}$:

- From **Cool**: *Slow* $\rightarrow$ stay Cool (reward +1); *Fast* $\rightarrow$ Warm or Cool (reward +2, each prob. 0.5).
- From **Warm**: *Slow* $\rightarrow$ Cool or Warm (reward +1, each prob. 0.5); *Fast* $\rightarrow$ Overheated (reward -10).
- From **Overheated**: absorbing, reward 0.

Choosing *Fast* yields higher immediate reward but risks a catastrophic transition. Evaluating whether *Fast* or *Slow* is better requires reasoning about long-term consequences — exactly what Theorem 1.3 and the Bellman Equation (next section) allow us to compute systematically.

1.5.8 Summary

- The **Markov property** (Def. 8) lets trajectories factorize (Lemma 1.1), avoiding the need to track full history.
- An **MDP** (Def. 9) formalizes environment dynamics and the agent's objective.
- **Discounting** with $\gamma < 1$ guarantees a finite return (Theorem 1.2).
- V_π and Q_π are related by a simple weighted sum (Theorem 1.3).
- Directly computing V_π, Q_π from their definitions is intractable in general — this motivates the **Bellman Equation**, covered next.

Instructor: Joun Won